

Python Reference / Golden Model

Full Project Specification

Systolic Array Matrix Multiplication Accelerator — Python companion model for verification, test generation, analysis, and later cocotb/regression use

Version: Python Golden Model v1.0

Purpose: Define a small, trustworthy Python reference implementation that reproduces the mathematical behavior of the completed C++ golden model and later serves as a verification/test-vector utility.

Core principle: Python computes *what* the accelerator should produce. It does not simulate *how* the systolic hardware produces it.

1. Project Context

The overall project is a 4×4 INT8 weight-stationary systolic matrix multiplication accelerator targeting the ZedBoard / Zynq-7020 (xc7z020clg484-1). The eventual design uses SystemVerilog RTL, UVM, AXI4-Lite control, AXI4-Stream data, and FPGA integration. The C++ golden model is the primary mathematical reference; Python is a companion reference and verification tool.

2. Exact Python v1.0 Scope

- Implement $A[M \times K] \times W[K \times N] = C[M \times N]$.
- Support rectangular matrices, not only square matrices.
- Support $1 \leq M, K, N \leq 16$.
- Use signed INT8 input/weight semantics and signed INT32 output/accumulation semantics.
- Provide known-answer tests, boundary tests, deterministic random tests, operation statistics, and PASS/FAIL comparison.
- Provide utilities that can later generate vectors for RTL/cocotb/UVM.
- Remain independent of the systolic architecture.

3. Explicitly Out of Scope

- No systolic-array cycle simulation.
- No PE simulation or FSM simulation.
- No weight-stationary movement simulation.
- No tiling or folding in the baseline reference.
- No AXI, DMA, BRAM, FPGA, Vivado, or Zynq modeling.
- No bias, activation, quantization, rounding, or saturation in v1.0.
- No GPU/BLAS/ML framework dependency for the core oracle.

4. Mathematical Definition

```
A[M×K] × W[K×N] = C[M×N]

C[i][j] = Σ(k=0..K-1) A[i][k] × W[k][j]

Required:
A.cols == W.rows

Output:
C.rows = A.rows
C.cols = W.cols
```

A is the input/activation matrix, W is the weight matrix, and C is the output matrix.

5. Numeric Model

Inputs: signed 8-bit range -128..127. **Product:** signed 16-bit semantics. **Accumulator/output:** signed 32-bit semantics.

Python integers are arbitrary precision. Therefore the model must explicitly validate the INT8 input range and document/enforce the intended result range rather than accidentally relying on Python's unlimited integer size. For the current $K \leq 16$ scope, INT32 is sufficient.

```
a = validate_int8(A[i][k])
w = validate_int8(W[k][j])
product = a * w
accumulator += product
C[i][j] = accumulator
```

Do not silently clamp values. Invalid values should produce a clear error.

6. Why Python Is a Companion

Python is useful for random generation, scripting, file I/O, regression management, data inspection, and later cocotb integration. The core implementation should remain a transparent triple-loop oracle rather than becoming an optimized numerical library.

7. Recommended Python Stack

Component	Purpose	v1.0
Python 3	Main language	Required
dataclasses	Statistics/config records	Recommended
typing	Public API type hints	Recommended
random	Reproducible random tests	Required
pathlib	File paths	Recommended
json	Optional vector interchange	Optional
argparse	Optional CLI regression	Optional
unittest	Automated tests	Recommended
NumPy	Later analysis/vector tooling	Optional
cocotb	Later RTL verification	Future

8. Recommended Project Structure

```
python_golden/
■■■ README.md
■■■ pyproject.toml # optional initially
■■■ golden/
■ ■■■ __init__.py
■ ■■■ matrix.py
■ ■■■ matmul.py
■ ■■■ stats.py
■ ■■■ validation.py
■■■ tests/
■ ■■■ test_matrix.py
■ ■■■ test_matmul.py
■ ■■■ test_edge_cases.py
■■■ scripts/
■ ■■■ run_regression.py
■ ■■■ generate_vectors.py
■■■ vectors/ # generated data
```

For the smallest first implementation, begin with **golden/matmul.py**, **golden/stats.py**, and **tests/test_matmul.py**. Add files only when they provide a clear benefit.

9. Matrix Representation

For v1.0, a list-of-lists representation is acceptable and deliberately readable.

```
A = [
[1, 2],
[3, 4],
]

W = [
[5, 6],
[7, 8],
]
```

Do not make NumPy's `matmul` the only oracle. A pure-Python reference keeps the arithmetic auditable and independent.

10. Matrix Validation

- Matrix must be non-empty.
- Every row must have the same number of columns.
- A and W values must be within -128..127 for the baseline.
- A.cols must equal W.rows.
- M, K, and N must be within 1..16.

Errors should clearly identify the problem, for example: *matrix dimension mismatch: A is 2x3, W is 4x2*.

11. Core `matmul` API

```
def matmul(A, W):
    validate_matrix(A, int8=True)
    validate_matrix(W, int8=True)

    M = len(A)
    K = len(A[0])
    N = len(W[0])

    if len(W) != K:
        raise ValueError("A.cols must equal W.rows")

    C = [[0 for _ in range(N)] for _ in range(M)]

    for i in range(M):
        for j in range(N):
            acc = 0
            for k in range(K):
                acc += A[i][k] * W[k][j]
            C[i][j] = acc

    return C
```

This is the conceptual implementation. Add validation and statistics cleanly; do not make the oracle unnecessarily complicated.

12. Statistics

```
multiplications = M * K * N
additions = M * N * (K - 1)
macs = M * K * N
primitive_ops = multiplications + additions
```

Recommended statistics record

```
from dataclasses import dataclass

@dataclass(frozen=True)
class MatmulStats:
    multiplications: int
    additions: int
    macs: int
    primitive_ops: int
```

13. Known-Answer Test

```
A = [[1, 2],
[3, 4]]
```

```
W = [[5, 6],
      [7, 8]]
expected = [[19, 22],
            [43, 50]]
```

Tests must compare values programmatically and fail when the result differs. Printing an expected value without asserting it is not a real regression test.

14. Test Matrix

Category	Examples
Basic	1×1, 2×2, 4×4
Rectangular	2×3 × 3×4; 4×8 × 8×4
Maximum	16×16 × 16×16
Zeros	All-zero matrices
Boundary	-128, 127
Mixed signed	Positive and negative values
Invalid	Empty, ragged, mismatch, out-of-range
Random	Seeded random dimensions and values
Exhaustive shapes	All 4096 M,K,N combinations

15. Deterministic Random Testing

```
import random

rng = random.Random(12345)
value = rng.randint(-128, 127)
```

Use a local Random instance. Record the seed on failures so every regression can be reproduced exactly.

16. Regression Runner

```
python scripts/run_regression.py --seed 12345 --count 1000
```

A failure report should include at least the seed, M/K/N dimensions, and enough information to reproduce the failing input.

17. C++ $\leftrightarrow$ Python Cross-Check

```
same test vector
█
██████████████████████
▼ ▼
C++ golden Python golden
█ █
▼ ▼
result_C result_P
█ █
██████████████████████
▼
compare equal
```

This is especially valuable because the C++ and Python implementations can expose mistakes in one another before either is used as a hardware reference.

18. File-Based Vectors

Later, JSON or another simple format can store M/K/N, A, W, expected C, seed, and statistics.

```
{
  "M": 2,
  "K": 2,
  "N": 2,
```

```
"A": [[1,2],[3,4]],
"W": [[5,6],[7,8]],
"expected": [[19,22],[43,50]]
}
```

These vectors can later be consumed by RTL simulation infrastructure.

19. Future cocotb Role

```
cocotb
#### generate A/W
#### drive DUT
#### wait for result
#### compare DUT output against Python matmul(A, W)
```

Keep the mathematical Python functions independent of cocotb so the same oracle can be used by scripts, regression, and future verification.

20. NumPy Policy

NumPy may later be used for convenient data handling, visualization, or an independent secondary checker. It should not replace the transparent pure-Python reference in v1.0.

21. Implementation Order

- Create the smallest Python project.
- Implement validation.
- Implement pure-Python matmul.
- Implement statistics.
- Write the 2×2 known-answer test.
- Add 1×1 and rectangular tests.
- Add INT8 boundary tests.
- Add invalid-input tests.
- Add deterministic random testing.
- Exercise all 4096 dimension combinations.
- Cross-check Python against the completed C++ golden model.
- Add optional JSON vector generation.
- Add a regression CLI.
- Integrate with cocotb only after the RTL exists.

22. Operation Count Reference

Shape	Multiplications	Additions	MACs	Primitive ops
1×1	1	0	1	1
2×2	8	4	8	12
3×3	27	18	27	45
4×4	64	48	64	112
8×8	512	448	512	960
16×16	4096	3840	4096	7936

23. Final Acceptance Criteria

- Python produces the same mathematical results as C++.

- INT8 inputs are validated.
- INT32 output semantics are documented.
- Rectangular matrices work.
- Known-answer and boundary tests pass.
- Invalid inputs fail clearly.
- Random tests are reproducible from a seed.
- All 4096 dimension shapes can be exercised.
- Statistics match the defined formulas.
- Python/C++ cross-check passes.
- No PE, systolic timing, AXI, DMA, or FPGA logic exists in the oracle.

24. Relationship to the Overall Project

[illegible]

25. Golden Rule

Keep the software model simpler than the hardware. The C++ and Python models should calculate the mathematical result directly. The RTL will implement the same mathematics through parallel, weight-stationary hardware.

26. First Python Milestone

```
python_golden/
├── golden/
│   ├── __init__.py
│   ├── matmul.py
│   ├── stats.py
│   └── tests/
│       └── test_matmul.py
```

First milestone: make the 2x2 known-answer test pass. Then add rectangular/boundary tests, deterministic random tests, and finally the C++/Python cross-check.